

Decode Labs Data Analytics Internship

Week 3 – Project 3: SQL for Data Analytics

Intern Name: Ifrah Aftab

Internship: Decode Labs Data Analytics Internship

Project Objective

The objective of this project was to learn and apply SQL (Structured Query Language) for data analysis using SQLite. The cleaned e-commerce dataset from Week 1 was imported into a SQLite database and analyzed using various SQL queries to retrieve, filter, sort, aggregate, and summarize data.

Tools Used

- DB Browser for SQLite
- SQLite Database
- Microsoft Excel (Cleaned Dataset)

Dataset Information

The dataset used in this project was the cleaned dataset prepared during Week 1 of the internship.

Dataset Features Included:

Columns: OrderID, Date, CustomerID, Product, Quantity, UnitPrice, ShippingAddress, PaymentMethod, OrderStatus, TrackingNumber, ItemsInCart, CouponCode, ReferralSource, TotalPrice

Total Records: 1,200

Database Name:

Week3_SQL Database.db

Table Name:

week3_data

Query 1 — Display First 10 Records

SQL Query

```
SELECT *  
FROM week3_data  
LIMIT 10;
```

Purpose: This query displays the first ten records from the dataset to verify that the data was imported successfully into SQLite.

SQL 1*														
1	SELECT *													
2	FROM week3_data													
3	LIMIT 10;													
4														
5														

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	ItemsInCart	CouponCode	ReferralSource	TotalPrice
1	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	Debit Card	Shipped	TRK37947903	7	SAVE10	Instagram	2,853.10
2	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	Online	Shipped	TRK91186779	3	SAVE10	Referral	302.7
3	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	Credit Card	Cancelled	TRK42903982	8	FREESHIP	Email	2,753.40
4	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	Debit Card	Returned	TRK62788070	5	SAVE10	Facebook	273.19
5	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	Online	Delivered	TRK29241424	8	SAVE10	Email	2,504.04
6	ORD200005	2023-10-23	C37249	Phone	2	245.86	934 Main St	Credit Card	Shipped	TRK72976927	4	SAVE10	Instagram	491.72
7	ORD200006	2025-06-17	C83492	Laptop	1	664.42	986 Main St	Gift Card	Returned	TRK96417362	6	SAVE10	Facebook	664.42

Execution finished without errors.
Result: 10 rows returned in 38ms
At line 1:
SELECT *
FROM week3_data
LIMIT 10;

Result: The query returned the first ten records of the e-commerce dataset.

Query 2 — Display OrderID, Product and UnitPrice

SQL Query

```
SELECT OrderID, Product, UnitPrice  
FROM week3_data;
```

Purpose: This query retrieves selected columns from the dataset instead of displaying all fields.

SQL 1*

1

2

3

4

5

6

SELECT *

FROM week3_data

LIMIT 10;

SELECT OrderID, Product, UnitPrice

FROM week3_data;

	OrderID	Product	UnitPrice
1	ORD200000	Monitor	570.62
2	ORD200001	Phone	151.35
3	ORD200002	Tablet	550.68
4	ORD200003	Chair	273.19
5	ORD200004	Printer	626.01
6	ORD200005	Phone	245.86
7	ORD200006	Laptop	664.42

Execution finished without errors.
Result: 1200 rows returned in 9ms
At line 4:
SELECT OrderID, Product, UnitPrice
FROM week3_data;

Result: The query returned OrderID, Product, and UnitPrice for all 1,200 records.

Query 3 — Products Above 500

```
SELECT *  
FROM week3_data  
WHERE UnitPrice > 500;
```

Purpose: To identify products with a unit price greater than 500.

SQL 1*

```
1 SELECT *
2 FROM week3_data
3 LIMIT 10;
4 SELECT OrderID, Product, UnitPrice
5 FROM week3_data;
6 SELECT *
7 FROM week3_data
8 WHERE UnitPrice > 500;
9
```

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	ItemsInCart	CouponCode	ReferralSource	TotalPrice
1	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	Debit Card	Shipped	TRK37947903	7	SAVE10	Instagram	2,853.10
2	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	Credit Card	Cancelled	TRK42903982	8	FREESHIP	Email	2,753.40
3	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	Online	Delivered	TRK29241424	8	SAVE10	Email	2,504.04
4	ORD200006	2025-06-17	C83492	Laptop	1	664.42	986 Main St	Gift Card	Returned	TRK96417362	6	SAVE10	Facebook	664.42
5	ORD200009	2023-11-21	C31946	Desk	4	509.38	102 Main St	Credit Card	Shipped	TRK33478363	6	SAVE10	Google	2,037.52
6	ORD200010	2023-12-29	C43443	Tablet	5	625.97	333 Main St	Credit Card	Returned	TRK98859248	9	WINTER15	Instagram	3,129.85
7	ORD200016	2023-01-24	C28976	Printer	2	533.81	914 Main St	Credit Card	Pending	TRK13194259	3	FREESHIP	Email	1,067.62

Execution finished without errors.
Result: 341 rows returned in 8ms
At line 6:
SELECT *
FROM week3_data
WHERE UnitPrice > 500;

Result: The query returned only high-priced products whose UnitPrice exceeded 500.

Query 4 — Only Phones

SQL Query

```
SELECT *
FROM week3_data
WHERE Product = 'Phone';
```

Purpose: To filter records and display only orders containing phones.

SQL 1*

```
1 SELECT *
2 FROM week3_data
3 LIMIT 10;
4 SELECT OrderID, Product, UnitPrice
5 FROM week3_data;
6 SELECT *
7 FROM week3_data
8 WHERE UnitPrice > 500;
9 SELECT *
10 FROM week3_data
11 WHERE Product = 'Phone';
12
```

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	ItemsInCart	CouponCode	ReferralSource	TotalPrice
1	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	Online	Shipped	TRK91186779	3	SAVE10	Referral	302.7
2	ORD200005	2023-10-23	C37249	Phone	2	245.86	934 Main St	Credit Card	Shipped	TRK72976927	4	SAVE10	Instagram	491.72
3	ORD200008	2025-04-02	C26817	Phone	2	134.28	904 Main St	Gift Card	Cancelled	TRK61042692	2	No Coupon	Email	268.56
4	ORD200023	2025-04-09	C42372	Phone	4	273.25	755 Main St	Debit Card	Cancelled	TRK36168083	5	FREESHIP	Referral	1,093.00
5	ORD200028	2023-11-21	C59823	Phone	4	160.07	568 Main St	Online	Pending	TRK65895539	5	No Coupon	Email	640.28
6	ORD200044	2025-04-21	C73225	Phone	2	550.18	996 Main St	Gift Card	Returned	TRK73848522	3	FREESHIP	Referral	1,100.36
7	ORD200053	2024-05-18	C67551	Phone	4	488.78	512 Main St	Online	Returned	TRK94992552	5	FREESHIP	Google	1,955.12

Execution finished without errors.
Result: 156 rows returned in 7ms
At line 9:
SELECT *
FROM week3_data
WHERE Product = 'Phone';

Result: The query returned all phone-related orders from the dataset.

Query 5 — Sort by Highest Price

SQL Query

```
SELECT *
FROM week3_data
ORDER BY UnitPrice DESC;
```

Purpose: To arrange products from highest price to lowest price.

12 SELECT *
13 FROM week3_data
14 ORDER BY UnitPrice DESC;
15

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	ItemsInCart	CouponCode	ReferralSource	TotalPrice
1	ORD200742	2024-11-30	C41974	Laptop	3	699.93	726 Main St	Cash	Returned	TRK48051468	7	SAVE10	Google	2,099.79
2	ORD201002	2025-06-19	C83911	Tablet	4	699.88	944 Main St	Cash	Pending	TRK33188106	5	WINTER15	Referral	2,799.52
3	ORD200908	2025-05-06	C35117	Desk	2	697.93	332 Main St	Credit Card	Cancelled	TRK27155709	2	SAVE10	Google	1,395.86
4	ORD200389	2023-12-17	C29703	Desk	4	697.5	558 Main St	Cash	Shipped	TRK40543461	7	FREESHIP	Referral	2,790.00
5	ORD201170	2023-08-03	C94257	Printer	2	697.19	762 Main St	Cash	Returned	TRK40220743	7	FREESHIP	Email	1,394.38
6	ORD201153	2024-12-16	C98317	Monitor	4	696.71	276 Main St	Debit Card	Shipped	TRK37172138	4	No Coupon	Instagram	2,786.84
7	ORD201149	2024-07-25	C27305	Chair	2	695.29	957 Main St	Credit Card	Pending	TRK28789916	6	FREESHIP	Instagram	1,390.58

Execution finished without errors.
Result: 1200 rows returned in 13ms
At line 12:
SELECT *
FROM week3_data
ORDER BY UnitPrice DESC;

Result: The most expensive products appeared at the top of the results

Query 6 — Sort by Lowest Price

SQL Query

```
SELECT *
FROM week3_data
ORDER BY UnitPrice ASC;
```

Purpose: To arrange products from lowest price to highest price.

15 SELECT *
16 FROM week3_data
17 ORDER BY UnitPrice ASC;

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	PaymentMethod	OrderStatus	TrackingNumber	ItemsInCart	CouponCode	ReferralSource	TotalPrice
1	ORD201161	2023-10-16	C32570	Phone	1	11.39	826 Main St	Cash	Cancelled	TRK35969968	6	WINTER15	Email	11.39
2	ORD200067	2024-12-09	C25587	Printer	4	12.1	440 Main St	Debit Card	Cancelled	TRK55105323	8	No Coupon	Google	48.4
3	ORD200863	2024-05-20	C98276	Phone	1	14.06	471 Main St	Credit Card	Pending	TRK74207282	5	SAVE10	Referral	14.06
4	ORD200251	2025-05-15	C93175	Monitor	4	14.69	814 Main St	Cash	Cancelled	TRK32995144	9	No Coupon	Email	58.76
5	ORD200473	2025-02-25	C28552	Chair	2	14.93	449 Main St	Online	Shipped	TRK71597160	4	No Coupon	Facebook	29.86
6	ORD200484	2024-05-29	C29092	Chair	4	15.01	767 Main St	Cash	Cancelled	TRK90455340	6	No Coupon	Facebook	60.04
7	ORD200240	2023-07-12	C14983	Tablet	1	17.24	255 Main St	Online	Pending	TRK93444157	2	FREESHIP	Instagram	17.24

Execution finished without errors.
Result: 1200 rows returned in 10ms
At line 15:
SELECT *
FROM week3_data
ORDER BY UnitPrice ASC;

Result: The least expensive products appeared first.

Query 7 — Count Total Orders

SQL Query

```
SELECT COUNT(*) AS TotalOrders
FROM week3_data;
```

18	SELECT COUNT(*) AS TotalOrders
19	FROM week3_data;
20	
	TotalOrders
1	1200

Result: Total Orders = 1,200

Query 8 — Total Quantity Sold

SQL Query

```
SELECT SUM(Quantity) AS TotalQuantity
FROM week3_data;
```

Result: Total Quantity Sold = 3,535

Insight: A total of 3,535 product units were sold across all orders.

20	SELECT SUM(Quantity) AS TotalQuantity
21	FROM week3_data;
22	
	TotalQuantity
1	3535

Query 9 — Average Product Price

SQL Query

```
SELECT AVG(UnitPrice) AS AvgPrice
FROM week3_data;
```

Result: Average Product Price = 356.41

22	SELECT AVG(UnitPrice) AS AvgPrice
23	FROM week3_data;
24	
	AvgPrice
1	356.41275

Query 10 — Number of Orders by Product

SQL Query

```
SELECT Product,  
COUNT(*) AS TotalOrders  
FROM week3_data  
GROUP BY Product;
```

Insight: Printers recorded the highest number of orders.

24 SELECT Product,
25 COUNT(*) AS TotalOrders
26 FROM week3_data
27 GROUP BY Product;|

	Product	TotalOrders
1	Chair	178
2	Desk	170
3	Laptop	173
4	Monitor	163
5	Phone	156
6	Printer	181
7	Tablet	179

Execution finished without errors.
Result: 7 rows returned in 23ms
At line 24:
SELECT Product,
COUNT(*) AS TotalOrders
FROM week3_data
GROUP BY Product;

Query 11 — Quantity Sold by Product

SQL Query

```
SELECT Product,  
SUM(Quantity) AS TotalSold  
FROM week3_data  
GROUP BY Product;
```

Insight: Chairs generated the highest quantity sold with 562 units.

```

28 SELECT Product,
29 SUM(Quantity) AS TotalSold
30 FROM week3_data
31 GROUP BY Product;
32

```

	Product	TotalSold
1	Chair	562
2	Desk	508
3	Laptop	535
4	Monitor	480
5	Phone	411
6	Printer	542
7	Tablet	497

```

Execution finished without errors.
Result: 7 rows returned in 27ms
At line 28:
SELECT Product,
SUM(Quantity) AS TotalSold
FROM week3_data
GROUP BY Product;

```

Query 12 — Average Price by Product

SQL Query

```

SELECT Product,
AVG(UnitPrice) AS AveragePrice
FROM week3_data
GROUP BY Product;

```

Result Summary:

Highest average prices were observed for:

1. Phone – 375.22
2. Tablet – 367.68
3. Monitor – 358.66

Insight: Phones had the highest average selling price among all product categories.

```

32 SELECT Product,
33    AVG(UnitPrice) AS AveragePrice
34 FROM week3_data
35 GROUP BY Product;

```

	Product	AveragePrice
1	Chair	355.660280898876
2	Desk	329.608117647059
3	Laptop	357.707052023121
4	Monitor	358.663619631902
5	Phone	375.217564102564
6	Printer	351.714917127072
7	Tablet	367.679106145251

Execution finished without errors.
 Result: 7 rows returned in 5ms
 At line 32:
 SELECT Product,
 AVG(UnitPrice) AS AveragePrice
 FROM week3_data
 GROUP BY Product;

Query 13 — Top Selling Products

SQL Query

```

SELECT Product,
SUM(Quantity) AS TotalSold
FROM week3_data
GROUP BY Product
ORDER BY TotalSold DESC;

```

Result Ranking:

1. Chair – 562
2. Printer – 542
3. Laptop – 535
4. Desk – 508
5. Tablet – 497
6. Monitor – 480
7. Phone – 411

Insight: Chairs were the top-selling products in terms of quantity sold.

```
36 SELECT Product,  
37 SUM(Quantity) AS TotalSold  
38 FROM week3_data  
39 GROUP BY Product  
40 ORDER BY TotalSold DESC;
```

	Product	TotalSold
1	Chair	562
2	Printer	542
3	Laptop	535
4	Desk	508
5	Tablet	497
6	Monitor	480
7	Phone	411

```
Execution finished without errors.  
Result: 7 rows returned in 5ms  
At line 36:  
SELECT Product,  
SUM(Quantity) AS TotalSold  
FROM week3_data  
GROUP BY Product  
ORDER BY TotalSold DESC;
```

Query 14 — Most Expensive Products First

SQL Query

```
SELECT Product,  
AVG(UnitPrice) AS AveragePrice  
FROM week3_data  
GROUP BY Product  
ORDER BY AveragePrice DESC;
```

Result Ranking:

1. Phone – 375.22
2. Tablet – 367.68
3. Monitor – 358.66
4. Laptop – 357.71
5. Chair – 355.66
6. Printer – 351.71
7. Desk – 329.61

Insight: Phones were the most expensive product category based on average price.

```

41 SELECT Product,
42    AVG(UnitPrice) AS AveragePrice
43 FROM week3_data
44 GROUP BY Product
45 ORDER BY AveragePrice DESC;

```

	Product	AveragePrice
1	Phone	375.217564102564
2	Tablet	367.679106145251
3	Monitor	358.663619631902
4	Laptop	357.707052023121
5	Chair	355.660280898876
6	Printer	351.714917127072
7	Desk	329.608117647059

```

Execution finished without errors.
Result: 7 rows returned in 5ms
At line 41:
SELECT Product,
AVG(UnitPrice) AS AveragePrice
FROM week3_data
GROUP BY Product
ORDER BY AveragePrice DESC;

```

Conclusion

This project successfully demonstrated the use of SQL for data analysis using SQLite. Various SQL commands including SELECT, WHERE, ORDER BY, COUNT, SUM, AVG, and GROUP BY were applied to analyze an e-commerce dataset. The analysis revealed key business insights regarding product sales, pricing patterns, and order distribution. Through this project, practical experience was gained in database management, query writing, and data-driven decision-making using SQL.